

LEVEL 16 — NumPy Broadcasting

A beginner-friendly guide with line-by-line examples and outputs.

1. What is Broadcasting?

Broadcasting means NumPy can perform operations between arrays of different shapes.

Example:

```
import numpy as np

a = np.array([10, 20, 30])
b = 5

result = a + b
print(result)
```

Output:

```
[15 25 35]
```

Line by line:

1. a is a 1D array with 3 values.
2. b is a single number, 5.
3. NumPy treats 5 as if it were [5, 5, 5].
4. It adds the values element by element.
5. Result = [10+5, 20+5, 30+5].

2. Broadcasting Rules

NumPy compares array shapes from right to left.

Two dimensions are compatible when:

1. They are equal, OR
2. One of them is 1.

Example:

```
A shape = (2, 3)
B shape = (3,)
```

Compare from the right:

```
3 with 3 -> equal -> OK
2 with no dimension -> allowed
```

Therefore broadcasting works.

Example:

```
A = np.array([[1, 2, 3],
               [4, 5, 6]])
B = np.array([10, 20, 30])

print(A + B)
```

Output:

```
[[11 22 33]
 [14 25 36]]
```

3. Scalar Broadcasting

A scalar is a single number.

Example:

```
a = np.array([[1, 2],
              [3, 4]])
```

```
result = a + 10
print(result)
```

Output:

```
[[11 12]
 [13 14]]
```

Explanation:

10 is broadcast to every element:

```
[[10, 10],
 [10, 10]]
```

Then NumPy adds the arrays.

4. 1D Broadcasting

A 1D array can be added to another compatible 1D array.

Example:

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
```

```
print(a + b)
```

Output:

```
[11 22 33]
```

Explanation:

$10+1 = 11$

$20+2 = 22$

$30+3 = 33$

Both arrays have shape (3,), so their shapes match.

5. 2D Broadcasting

Example:

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([[10, 20, 30],
              [40, 50, 60]])
```

```
print(a + b)
```

Output:

```
[[11 22 33]
 [44 55 66]]
```

Both arrays have shape (2, 3), so NumPy adds matching elements.

6. Row Broadcasting

A row vector has shape (1, 3).

Example:

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
row = np.array([10, 20, 30])
```

```
print(a + row)
```

Output:

```
[[11 22 33]
 [14 25 36]]
```

The row [10, 20, 30] is used for every row:

```
[1, 2, 3] + [10, 20, 30]
```

```
[4, 5, 6] + [10, 20, 30]
```

7. Column Broadcasting

A column vector has shape (2, 1).

Example:

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
```

```
column = np.array([[10],
                   [20]])
```

```
print(a + column)
```

Output:

```
[[11 12 13]
 [24 25 26]]
```

The 10 is broadcast across the first row.

The 20 is broadcast across the second row.

Conceptually:

```
[[10, 10, 10],
 [20, 20, 20]]
```

8. Broadcasting Error

Broadcasting fails when shapes are incompatible.

Example:

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
```

```
b = np.array([10, 20])
```

```
print(a + b)
```

Output:

```
ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```

Why?

Compare from the right:

3 and 2 -> not equal

Neither value is 1

Therefore broadcasting is not possible.

9. Broadcasting with Arithmetic

Broadcasting works with +, -, *, and /.

Example:

```
a = np.array([[10, 20, 30],
              [40, 50, 60]])
```

```
b = np.array([1, 2, 3])
```

```
print(a - b)
print(a * b)
```

Output:

```
[[ 9 18 27]
 [39 48 57]]
```

```
[[ 10 40 90]
 [ 40 100 180]]
```

The 1D array is broadcast across both rows.

10. Broadcasting with Functions

NumPy functions can also use broadcasting.

Example:

```
a = np.array([[1, 4, 9],
              [16, 25, 36]])
```

```
b = np.array([1, 2, 3])
```

```
result = np.sqrt(a) + b
print(result)
```

Output:

```
[[ 2.  4.  6.]
 [ 5.  7.  9.]]
```

Line by line:

1. `np.sqrt(a)` calculates the square root of every value.
2. `sqrt(a)` becomes:

```
[[1., 2., 3.],
 [4., 5., 6.]]
```
3. `b = [1, 2, 3]` is broadcast to both rows.
4. The arrays are added.

11. Quick Shape Practice

Example 1:

```
A shape = (2, 3)
B shape = (3,)
Compatible? YES
```

Example 2:

```
A shape = (2, 3)
B shape = (2, 1)
Compatible? YES
```

Example 3:

```
A shape = (2, 3)
B shape = (2,)
Compatible? NO
```

Remember:

Compare dimensions from right to left.

Equal -> OK

One is 1 -> OK

Missing dimension -> OK

Otherwise -> Error.

12. Summary

Broadcasting allows NumPy to work with arrays of different but compatible shapes without manually copying data. The key rule is to compare shapes from right to left: dimensions must be equal or one of them must be 1. Scalars can broadcast to any shape. This makes arithmetic and many NumPy functions fast and convenient.